

Real-World Baselines Challenge: Dynamic Train Rescheduling under Stochastic Perturbations

Manuel Meyer¹, Christian Eichenberger¹, Darshan Makwana², Venkata Avinash Guntupalli³, Roman Liessner⁴, Manuel Schneider¹, Anton Fuxjäger⁵, Adrian Egli⁶, Daniel Boos⁶, and Ricardo Jorge Bessa⁷

`competition@flatland-association.org`

¹FLATLAND Association, Switzerland, ²IIT Bombay, India, ³Independent Researcher, ⁴DB InfraGO AG, Germany, ⁵enliteAI, Austria, ⁶SBB Swiss Federal Railways, Switzerland, ⁷INESC TEC, Portugal

Abstract. In this competition, participants tackled the problem of dynamic train rescheduling under stochastic perturbations. The competition used the open-source Flatland environment and supported both operations research (OR) and reinforcement learning (RL) approaches. Compared to prior Flatland competitions, this competition increased real-world relevance via multi-stop journeys with time windows at stops, controlled routing flexibility, manually curated network topologies and multiple disruption types (departure delays, train breakdowns, and infrastructure failures). The winning solutions, outperforming the given deadlock avoidance heuristic baseline, implemented temporal-spatial OR algorithms with initial planning. No RL solution could match the RL baseline which combines learned policies with rule-based safeguards.

Keywords: reinforcement learning · operations research · vehicle rescheduling problem · railway traffic management

1 Challenge Description

1.1 Background and Novelty

In real-world railway operations, trains follow schedules planned in advance at multiple levels of granularity. Despite time reserves built into these plans, they cannot always be maintained due to numerous perturbations such as engine breakdowns or externally induced delays. Therefore, the operational plan must be updated through reordering and cancellation decisions. This is known as the vehicle rescheduling problem (VRSP) [1]. Even small disturbances (e.g., delayed door closure, slow passenger boarding) can propagate and cascade forward in both space and time considerably.

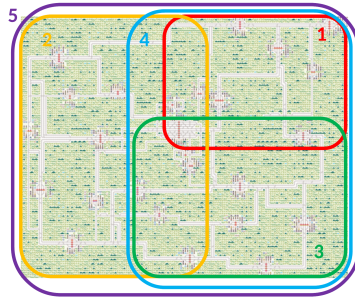


Fig. 1. The competition topology of size (120 x 150) with 28 stations and its subdivisions into 5 *scenes* as provided to participants.

The Flatland framework [2] formulates the VRSP as a research problem, and has been used for previous challenges [3,4]. Participants submit solutions that guide trains on a given infrastructure map according to a mesoscopic timetable, i.e., a timetable that does not specify fully conflict-free trajectories in space and time, but only time-windows at intermediate stops and destinations. Trains are grouped into categories characterized by their speed. No variable speeds are used in this competition, so trains run at max. speed or 0.

Novel in this competition, Flatland supports (i) journeys with intermediate stops and (ii) routing flexibility, i.e., an intermediate stop is considered served whenever the train stops at any of the tracks associated with that station. Furthermore, there are new malfunction classes. In addition to random wayside breakdowns of trains, (iii) random malfunctions at departures from stops causing trains to be delayed and (iv) infrastructure failures are introduced. Finally, (v) the competition runs on a manually curated static topology instead of randomly generated environments. For the full technical description of the competition setup (data, evaluation, logistics), see. [5].

The introduction of these novelties is motivated by creating a simulation setting closer to real-world problems. Special interest lies in determining whether reinforcement learning (RL) solutions benefit from a fixed topology, i.e., if they can exploit the recurring structure of the environment. For operations research (OR) approaches, the main questions remain how to compromise on the computational budget for planning and re-planning and how well they scale with number of trains and malfunctions. This competition also aims to find new state-of-the art baselines and explore their strategies to bring trains to their intermediate stops and destinations on time. Increasingly tighter schedules and number and severity of malfunctions test their limits.

1.2 Data

The evaluation set is structured into 7 levels. *Levels*: consist of 5 scenarios corresponding to the 5 scenes (cf. Figure 1) with the exception of level 6, where all five scenarios use scene 5. *Malfunctions*: Levels 0–2 have no malfunctions; Level 3 adds wayside breakdowns; Levels 4 and 5 add departure delays; Level 6 adds infrastructure breakdowns. *Number of agents*: Levels have increasing number of agents, from 8–28 in Level 0 to 532 in Level 6. *Generation*: a pipeline with hidden configuration multiplies initial lines and timetables with parametrized amount, offset and malfunctions per level.

For training curricula, participants were provided with the single competition topology shown in Figure 1 and a subset of 6 lines from the evaluation set.

1.3 Evaluation

The parametrization of the reward function incentivizes to (i) put trains on the map instead of canceling their journeys, (ii) serve intermediate stops even when delayed and at the risk of departure delays, (iii) arrive at the destination even when delayed, and (iv) heavily discourages collisions.

For each scenario a normalized reward is calculated, and the sum of these rewards over all completed scenarios yields the submission score

$$S_{\text{submission}} = \sum_{l=0}^6 \sum_{s=1}^5 \left(\sum_{a=1}^{a_{l,s}} \frac{\max\{r_a^{l,s}, -m_{l,s}\}}{m_{l,s} a_{l,s}} + 1 \right), \quad (1)$$

where $m_{l,s}$ and $a_{l,s}$ are the max. episode steps and number of agents for scenario s of level l , and $r_a^{l,s} < 0$ is the total reward for agent a in level l and scenario s . The maximum score is 35. For details on evaluation and logistics, see [5].

2 Solutions

In the non-RL track, two solutions were able to outperform the given baseline, while in the RL track none beat the baseline from the organizers. Table 1 provides an overview of the solutions described in this section.

Table 1. Leaderboard combining both RL and non-RL tracks.

Track	Rank	Participant	Solution	Score
non-RL	1	darshanmakwana412	<i>v4-sipp-locks</i> [eval code]	20.84
non-RL	2	Avinash837	<i>flatland_v3</i> [eval code]	12.49
non-RL	–	organizers	<i>deadlock-avoidance</i> (DLA) [eval code]	10.73
RL	–	organizers	<i>adaptive-completion</i> [eval code]	9.42

2.1 Winner: darshanmakwana412 / *v4-sipp-locks*

The winning submission, *v4-sipp-locks*, adapts safe interval path planning (SIPP) to the multi-agent railway setting. Its two central ingredients are a *shared reservation table* over grid resources and *directional corridor locks*. The reservation table records for each resource (grid cell) the time intervals during which it is occupied by an already planned agent. Whenever a plan traverses a single-track corridor, it additionally locks the corridor for its direction and time window, such that opposing plans are forced to schedule around it (cf. Figure 2). Same-direction platooning remains admissible, which preserves throughput on the busiest tracks.

Agents are planned sequentially in a prioritized fashion. The priority order is given by increasing *slack*, defined as the difference between the latest arrival time at the target stop and the earliest feasible arrival when running at the agent’s maximum speed along its shortest path. Agents with the tightest slack are planned first, since they have the least room to accommodate detours or waits. Each agent is then routed through the reservation table with a cost-priority queue that expands safe intervals in the spirit of SIPP, with the cost combining travel time, waiting time, and deadline violation.

The approach is completed by a lightweight replanning loop: when a newly introduced malfunction invalidates parts of the current reservation table, the

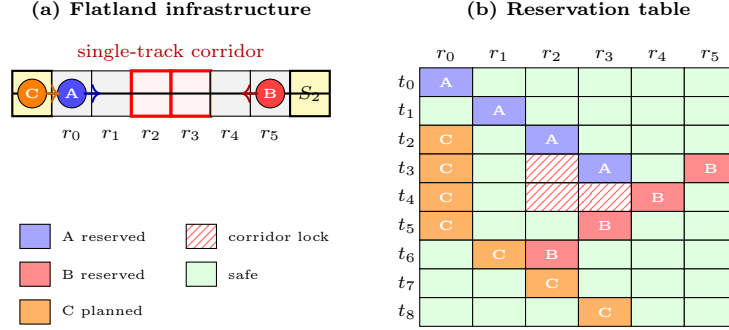


Fig. 2. Illustration of SIPP-based planning with corridor locks on a Flatland track. (a) A single-track corridor (r_2, r_3 , red border) connects stations S_1 and S_2 (yellow). Agent A (blue, $\rightarrow$) traverses the corridor from left to right, agent B (red, $\leftarrow$) approaches from the opposite direction, and a new agent C (orange, $\rightarrow$) shares A’s direction. (b) The reservation table is discretized per timestep: each row t_i specifies, for every resource r_j , exactly one state (occupied by an agent, locked by A’s corridor traversal, or free/safe). Agent C is planned through the remaining safe intervals: it waits in r_0 during $t_2 \dots t_5$ until the corridor lock releases and B has passed, then advances toward S_2 .

affected agents are re-inserted into the priority queue and replanned against the updated table, while unaffected plans are retained. This locality of replanning keeps the per-step cost tractable even on the densest scenarios.

2.2 Runner-Up: Avinash837 / *flatland_v3*

The core of the runner-up solution *flatland_v3* is a centralized, reservation-based dispatcher. The problem is treated as one-shot multi-agent path finding with prioritized planning where each train is assigned a timed path protected by space-time reservations on cells and edges.

Routing is performed directly on a pre-processed graph. For each train, up to 4 candidate routes are generated using penalized Dijkstra search. After a route is found, each cell on that route receives an additional cost penalty, encouraging subsequent searches to use different corridors. Routes lead directly to the final target. The value of completing a train outweighs the relatively small penalty associated with skipping intermediate stops, while substantially longer multi-stop routes can prevent trains from receiving feasible schedules.

Trains are planned fastest-first. Faster trains traverse and release infrastructure sooner, leaving additional capacity for trains scheduled later. The planner tests every candidate route with a fixed set of departure offsets, inserting waits at intermediate cells at need. The first candidate that reaches the target is added to the shared reservation table. A train that cannot be scheduled is delayed at the depot rather than released on to the network.

The executor performs occupancy-based arbitration, rather than performing a full joint space-time search or iterative global refinement. A move is granted

when the destination cell is empty or when its current occupant has itself been granted permission to move out during the same step. This allows nose-to-tail queues to advance together.

2.3 Non-RL Baseline: *deadlock-avoidance-heuristic*

The non-RL baseline *deadlock-avoidance-heuristic* (DLA) tries to prevent deadlocks. In train operations, deadlocks (trains standing face-to-face) need to be prevented ahead of time, they cannot easily be resolved after the fact. In Flatland, deadlocks cannot be resolved at all as there is no action to move backward.

The heuristic has complete knowledge of the current state of all trains and controls them centrally. It starts with each train choosing a path unilaterally when the episode starts, using shortest-path routing between the source and target stations. The basic idea is then for each train: *Do not enter/move if there is no free space between my path and any oncoming train's path* (a train is oncoming if it is on my path and running towards me). As long as (i) trains “see” each other and (ii) there are no further trains blocking them, there is at least one non-overlapping segment between two trains where they can evade each other. Otherwise, deadlocks can occur.

2.4 RL Baseline: *adaptive-completion*

The RL baseline *adaptive-completion* combines a learned per-agent policy with a few lightweight rule-based guards that keep the trains safe in situations the policy does not resolve on its own.

The learned core is a decentralized actor-critic. Each agent observes a local feature vector with an action mask over the five Flatland actions appended to it. The network is a small multi-layer perceptron (three hidden layers of 128 units, tanh activations) with separate policy, value, risk and action-value heads, and it is trained with PPO on a curriculum of increasingly hard scenarios.

Around this policy guards that act on the proposed actions are added. A look-ahead re-ranking step (about 45 cells) reconstructs the route prefix implied by each action and, using deadline slack, detects upcoming head-on conflicts and steers the affected agents onto alternative moves. A deadlock-avoidance fallback handles the densest scenarios.

The policy learns a usable end-to-end controller and scales to the full agent count at negligible inference cost. The central challenge that remains is credit assignment: a purely ego-centric reward carries too little information about the joint outcome, while a fully global reward is too noisy to guide an individual agent. The deadlocks that decide the harder levels are properties of whole corridors and only become visible several steps ahead, which a local observation with a short effective horizon cannot capture.

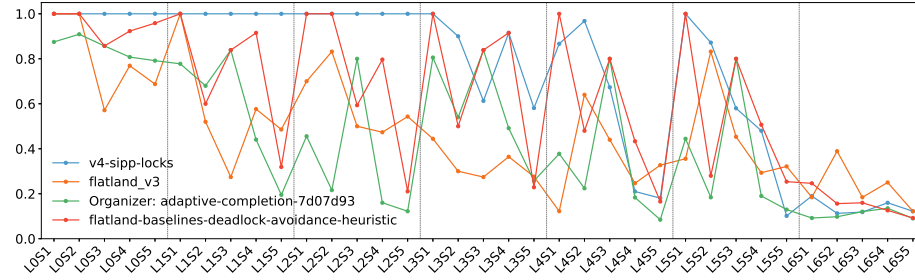


Fig. 3. Success rates per scenario of the 4 solutions.

3 Comparison and Analysis of Solutions

Table 2 shows a high-level comparison of the solutions. Both participant solutions follow the tradition of the winning solutions of earlier Flatland challenges [3,4].

Table 2. Comparison of the solutions along 3 dimensions: whether an initial microscopic plan is produced before the simulation is started, i.e., a conflict-free plan at the time-step and cell level; whether intermediate stops are considered; whether spatial alternatives or only the timing of fixed spatial routes are considered during rescheduling.

Submission	Initial Microscopic Plan	Intermediate stops	Re-planning
<i>v4-sipp-locks</i>	✓	✗	temporal-spatial
<i>flatland_v3</i>	✓	✗	temporal
<i>dla</i>	✗	✗	temporal
<i>adaptive-completion</i>	✗	✗	temporal-spatial

Comparing success rates, i.e., the rate of trains arriving at their destination, across all scenarios as shown in Figure 3, one can see that *v4-sipp-locks* reaches 100% for all scenarios without malfunctions. For the last level, *flatland_v3* shows the highest success rate (23%) as well as the highest normalized score (1.29). Further idiosyncrasies are higher success rates for certain scenarios regardless of the level (e.g., S1 for *DLA*, S3 for *adaptive-completion*, S2 for *flatland_v3*) as well as overall higher success rates for specific scenarios (e.g., L4S3, L5S1, L5S3). Reasons for these correlations might be the specifics of the corresponding scenes and levels that are exploited differently (e.g., more alternative routes, longer tracks between stations) or similarly (e.g., (un)favorable malfunction seeds).

The detailed rewards and their rates contributing to the overall reward per level are shown in Figure 4. There are three important observations: (i) collision penalty dominates (with the exception of *v4-sipp-locks* for malfunction free levels), (ii) intermediate stops are completely neglected (the shares differ because the total normalized rewards differ between solutions), and (iii) target not reached rate as well as cancellations are more pronounced for higher levels.

Observation (iii) is easily understood, since the levels are more difficult and it might pay out to cancel some trains to reduce overall delay or increase success rate. Observations (i) and (ii) need some explanation. They are related to an

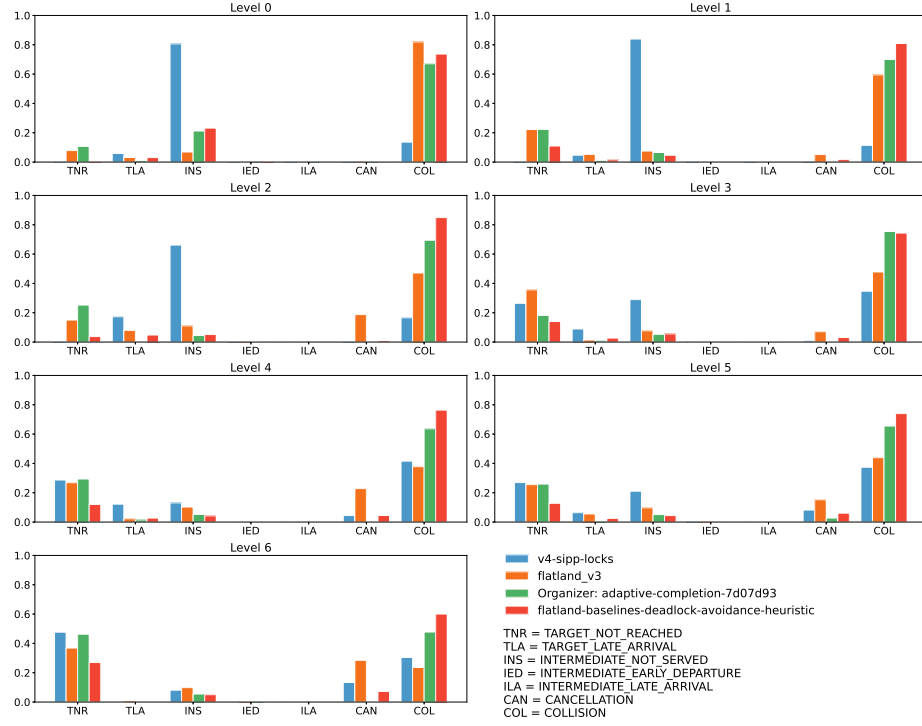


Fig. 4. Comparing detailed rewards as shares of the total normalized reward to highlight the main contributions to the penalties per solution and level.

error in the reward implementation, penalizing not only collision whenever the simulation had to intervene as intended, but also whenever the state of an agent changed from MOVING to STOPPED as fully intended by the underlying policy. This implies that it is advantageous to omit intermediate stops altogether. Another implication of the wrongfully penalized state transition is that platooning, i.e., driving in a line one train behind the other, can be very costly. As the trains can have different speeds, slower trains at the front force faster ones behind them to continuously start and stop moving, thus triggering the penalty at each stop.

However, the winning solution internalized the unintended penalty and therefore showed that it didn't prevent meaningful solutions overall.

4 Conclusion

All conclusions are conditioned by the defect in the reward implementation, (Section 3). The results, therefore, reflect the challenge de facto without the intermediate stops part of the problem. Nevertheless, some interesting conclusions can be drawn.

The leaderboard gives no evidence that RL benefits from the fixed topology, however, the RL track was too sparsely populated to treat this question as

settled. The winning dispatchers scaled to the largest scenarios (532 agents) well within the time budget, controlling cost through prioritized planning, a cached topology, and — for the winner — malfunction-localized replanning rather than raw search. The 100% success rate for disruption-free levels illustrates an efficient planning algorithm even if not all trains reach their destination on time. Level 6 was designed to test the limits of the solutions. However, the decline in success rate within levels 4 and 5 already suggests an acceptable limit was reached earlier. In addition to malfunctions which are parametrized per level, the number of agents and the tight schedules are the limiting factors in this setting.

The defect that made stops uneconomical and all submissions to neglect intermediate stops (cf. Figure 4) necessitates future work to explore state-of-the-art approaches including intermediate stops. With the open-source data pipeline implemented for this competition, a tool exists to scale the density of trains and malfunctions arbitrarily in order to explore the limits in a close to real-world topology in future work on the state-of-the-art of dynamic train rescheduling.

Acknowledgments. This competition has been organized as part of AI4REALNET which has received funding from European Union’s Horizon Europe Research and Innovation programme under the Grant Agreements No 101119527 and from the Swiss State Secretariat for Education, Research and Innovation (SERI). Views and opinions expressed are those of the authors only and do not necessarily reflect those of the European Union and SERI. Neither the European Union nor the granting authority can be held responsible for them.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

Authors’ Contributions Statement. ¹ Writing – original draft: M.M. and Ch.E. Writing – Description winning solution: D.M. Writing – Description runner-up solution: V.A.G. Writing – RL baseline: R.L. Writing – review & editing: all authors.

References

1. Li, J.Q., Mirchandani, P.B., Borenstein, D.: The vehicle rescheduling problem: Model and algorithms. *Networks* **50**, 211–229 (2007)
2. Mohanty, S., et al.: Flatland-RL: Multi-agent reinforcement learning on trains. arXiv preprint arXiv:2012.05893 (2020)
3. Nygren, E., et al.: Flatland Challenge: Multi-agent reinforcement learning on trains. *AIcrowd Competition* (2019)
4. Laurent, F., et al.: Flatland Competition 2020: MAPF and MARL for efficient train coordination on a grid world. In: *NeurIPS 2020 Competition and Demonstration Track*, pp. 275–301. PMLR (2021)
5. Flatland Book: ECML 2026 Challenge. <https://flatland-association.github.io/flatland-book/challenges/ecml2026.html>, last accessed 2026/07/03

¹ Following the CRediT taxonomy, <https://credit.niso.org/>